

SECBench: A Benchmarking Framework for Stream Processing Systems in the Sensor-Edge-Cloud Continuum

Taha Tekdogan¹(✉), Lukas Schwerdtfeger¹, Tilmann Rabl², Steffen Zeuch¹,
and Volker Markl^{1,3}

¹ BIFOLD, Technische Universität Berlin, Germany

² Hasso Plattner Institute, Germany

³ DFKI GmbH, Germany
tekdogan@tu-berlin.de

Abstract. Stream processing systems (SPSs) process large volumes of data in real time by distributing computation across multiple machines. Many established SPS deployments focus on centralized cloud environments. However, as more data are generated outside of the cloud, forwarding all the raw data can increase the network load and limit the responsiveness of applications. Modern SPSs address this limitation by distributing the computation across sensor, edge, and cloud resources, forming the *sensor-edge-cloud (SEC) continuum*. Although the performance of SPSs has been studied extensively in cloud-only settings, benchmarks provide limited support for evaluating SPSs under the hardware heterogeneity and network diversity of SEC deployments. To address this gap, we propose *SECBench*, a benchmarking framework for evaluating SPSs in the SEC continuum. SECBench generates workloads, executes benchmark runs, and measures SPS performance while deploying systems in configurable SEC environments that model hardware and network properties. We demonstrate the applicability of SECBench using two real-world use cases with different workload and deployment characteristics. SECBench provides a foundation for studying how SEC configurations affect SPS performance and for comparing cloud-only and SEC processing models in reproducible benchmark environments.

Keywords: Stream processing · Data management · Benchmarking · Sensor-edge-cloud continuum

1 Introduction

The number of connected Internet of Things (IoT) devices worldwide is projected to grow from 21 billion in 2025 to 39 billion by 2030 [50]. These devices continuously produce large volumes of real-time sensor data. Many IoT applications require response times on the order of milliseconds [6, 43]. Despite that, established solutions rely predominantly on centralized cloud processing, where message brokers collect data coming from distributed sources and forward them to cloud servers [20]. As data volumes grow, this design can stress the network

bandwidth, increase the load on centralized cloud resources, and make it difficult to meet the latency requirements of IoT applications [31, 48].

At the same time, end devices such as sensors, wearables, or smart appliances have enhanced their computing capabilities in recent years [48]. These advancements have enabled the utilization of their resources by pushing computation closer to data sources [27, 45, 62]. As a result, processing can be distributed across multiple layers of the deployment, from sensors to cloud servers, forming the *sensor-edge-cloud continuum (SEC)* [20]. This emerging computing paradigm has been adopted in many domains, ranging from agriculture to smart manufacturing [20, 31]. Offloading computation closer to data sources can reduce network traffic, diminish the load on central servers, and improve data privacy by limiting the amount of raw data that leaves the edge [56].

The need to process continuously generated data with low latency makes stream processing essential for many IoT applications [29]. Stream processing systems (SPSs) process unbounded data streams efficiently [16]. However, their performance depends on many factors such as the workload, deployment environment, and resource constraints. Therefore, it is important to choose an SPS according to the application requirements, but also to understand the performance characteristics and limitations of the chosen SPS [61]. Benchmarks provide the foundation for such assessments by defining evaluation metrics and workloads that reflect real-world scenarios and stress the limits of SPSs [59]. Several benchmarks evaluate the performance of SPSs in various settings [8, 13, 16, 26, 30, 54, 59]. Although these benchmarks provide valuable insights, their setups focus on cloud-only environments and do not capture edge processing within the SEC [59]. To evaluate the end-to-end behavior of an SPS running in the SEC continuum, a benchmark should consider the properties of the underlying infrastructure on which processing takes place.

In particular, SEC deployments combine two characteristics that are not yet jointly considered as first-class, configurable properties in SPS benchmarks. The first characteristic is the network structure, as the SEC continuum organizes devices in a layered hierarchy, from sensor devices to cloud servers [34, 57]. A benchmark for this setting must model the network properties, such as topology, bandwidth, and latency. The second characteristic is the hardware heterogeneity, where sensor devices, edge infrastructures, and cloud servers differ in their computational capabilities [56]. As a result, performance of an SPS running in the SEC continuum depends not only on the workload or the stream processing engine, but also on the underlying infrastructure [12, 14]. Network topology, link latency and bandwidth, hardware capacity, and operator placement can all influence the performance [14, 30]. A benchmark must account for these factors to evaluate how an SPS behaves across these heterogeneous deployments. These challenges motivate a benchmarking framework that models the characteristics of the continuum and supports benchmark deployments for the SEC continuum.

In this paper, we present SECBench, a benchmarking framework for SPSs in the SEC continuum. SECBench extends the environment of the system under test (SUT) beyond cloud-only settings, and models edge and sensor resources

as part of the benchmarking environment [30]. It also adapts established evaluation metrics to end-to-end SEC deployments, where data are generated at distributed sources and processed across sensor, edge, and cloud nodes. Based on these extensions, SECBench supports complete benchmark runs in SEC deployments by combining configurable environments, workload generation, and performance measurement. We demonstrate the applicability of SECBench with two real-world use cases with representative workload and deployment characteristics. Our experimental results show that SECBench can deploy realistic SEC environments and capture SPS behavior and performance characteristics in diverse deployment settings, which highlight the need to evaluate SPSs across the SEC.

In summary, we make the following contributions:

- We introduce SECBench, a benchmarking framework for evaluating SPSs in the SEC continuum, which orchestrates workload generation, SUT execution, and performance measurement in configurable SEC environments.
- We identify network topology and hardware heterogeneity as key environment characteristics of the SEC continuum, and model them in SECBench.
- We extend the definition of SUT and evaluation metrics to support end-to-end evaluation of SPSs across sensor, edge, and cloud nodes.
- We demonstrate the applicability of SECBench through two real-world use cases with different workload and deployment characteristics.
- We assess the fidelity of SECBench by comparing its results with bare-metal executions, and show that it captures performance behavior and environmental characteristics with small overhead.

The remainder of this paper is organized as follows. Sec. 2 provides the motivation, and Sec. 3 introduces SECBench, its underlying methodology, and its main components. Sec. 4 defines the evaluation metrics, presents two use cases with corresponding experiments, and presents a validation experiment. Sec. 5 discusses the related work, and Sec. 6 concludes the paper.

2 Motivation

Stream processing systems (SPSs) continuously ingest and process unbounded data streams to produce low-latency results [5, 51]. Modern SPSs scale by partitioning input events, execute operators in parallel, and exchange intermediate results across distributed workers [9, 60]. However, this scalability addresses only a part of the challenge. Contemporary applications gather data from large numbers of distributed data sources, such as IoT sensors, cameras, and mobile devices [48]. When these data sources send continuous, high-volume data streams to cloud servers, bottlenecks can shift from the SPS to the network infrastructure or message brokers that ingest and deliver the data [46, 48].

The *sensor-edge-cloud (SEC) continuum* combines resources from sensors, edge devices, and cloud servers into a unified and distributed deployment [2, 20]. Instead of forwarding all raw data to the cloud, applications can process data closer to their sources and forward only intermediate or final results upstream.

This processing model can reduce network traffic, diminish the load on cloud servers, and improve responsiveness in latency-sensitive applications [12, 46].

The SEC continuum introduces two characteristics that distinguish it from cloud-only deployments. The first characteristic is its *network topology*, as the continuum consists of geographically distributed devices connected by heterogeneous links [20]. They often form centrally hierarchical topologies, where sensor devices connect to nearby edge infrastructures, while edge infrastructures connect to a central server [2]. These connections differ in latency, bandwidth, and reliability because they rely on different network technologies and cover different distances [7]. The second characteristic is *hardware heterogeneity*, as devices in the SEC continuum have different compute, memory, and storage capacities [20]. These capacities can vary substantially, ranging from constrained sensor devices to cloud servers. These characteristics influence the behavior and the performance of SPSs, together with the workload and the design of the underlying SPS [12, 14].

These properties change what a benchmarking framework must support. In the SEC continuum, such a framework must account for the distributed environment in which processing takes place. Data may originate from geographically distributed data sources, traverse heterogeneous network links, and be processed on nodes with different capacities. A benchmarking framework for SPSs in this setting must therefore enable users to configure not only the workload, but also the network topology and the hardware heterogeneity that characterize the SEC deployments [7, 20]. Existing SPS benchmarks provide valuable workloads and evaluation metrics, but they do not fully capture the environment characteristics of the SEC continuum. PDSPBench [1] is the closest to our work as it considers heterogeneous execution environments, but it does not target end-to-end SEC deployments where geographically distributed sources, heterogeneous network links and hardware capacities together affect the performance of SPSs. We discuss existing SPS benchmarks in more detail in Sec. 5. These limitations motivate a new benchmarking framework that can construct SEC environments, generate workloads to deployed SUTs, and measure performance of them under configurable deployment conditions.

3 SECBench Framework

SECBench is an end-to-end benchmarking framework for SPSs in the SEC continuum. It extends cloud-centric benchmarks by modeling the characteristics of the continuum that affect performance. SECBench uses these characteristics to *emulate*⁴ SEC deployments with configurable network and hardware properties. It then integrates this constructed environment into an end-to-end benchmarking flow that generates workloads, deploys the SUT, and measures performance. In the remainder of this section, we introduce the environment characteristics that we use to represent the SEC continuum (Sec. 3.1), present SECBench’s

⁴ We use Containernet [40] to emulate distributed environments, which we discuss in Sec. 3.3.

benchmarking flow and its components (Sec. 3.2), describe how we construct the environment (Sec. 3.3), and describe how to run an existing SPS as the SUT within SECBench (Sec. 3.4).

3.1 SEC Environment Characteristics

As we discussed in Sec. 2, the performance of the SPSs in the SEC continuum not only depends on the workload and the system design, but also on the environment in which the system is running. This subsection describes the characteristics of the deployment environment that SECBench makes configurable in the benchmark setup, i.e., *hardware heterogeneity* and *network topology*. SECBench models these characteristics, and enables users to instantiate diverse continuum settings in a controlled and reproducible manner. In the following, we describe how SECBench represents each environment characteristic.

Hardware Heterogeneity. SECBench models the *hardware heterogeneity* to capture the device heterogeneity of SEC deployments. It parametrizes each node with a configurable resource profile. During the deployment of experiments, SECBench assigns CPU, main memory, and storage limits to Docker hosts, and enforces these limits so that each host operates within the specified resource constraints. Users configure these constraints in the **hardware** section of its configuration file. Listing 1.1 shows an example configuration for hardware heterogeneity with two nodes with different resource profiles. For example, **node1** has 64 CPU cores, 512 GB of main memory, and 2 TB of storage, representing a cloud instance such as an AWS EC2 instance [55]. In contrast, **node2** has 4 CPU cores, 8 GB of main memory, and 16 GB of storage, representing a lightweight edge device such as a Raspberry Pi [44]. The configuration assigns each node an identifier, and specifies its resource parameters with **cpu**, **memory**, and **storage**, which denote the number of allocated CPU cores, the amount of assigned main memory, and allocated amount of storage, respectively. While deploying the setup, SECBench constructs the corresponding environment by using this configuration, which we depict in Fig. 1.

Listing 1.1: Hardware and network configurations in SECBench

```
hardware:
- id: node1
  cpu: 64 # no of cores
  memory: 512g
  storage: 2048g
- id: node2
  cpu: 4
  memory: 8g
  storage: 16g
network:
  topology: custom
  - link: node1:node2
    latency: 15 # ms
    bandwidth: 200 # Mbps
```

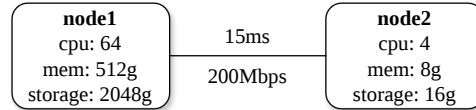


Fig. 1: Constructed environment by SECBench using the configurations from Listing 1.1.

Network Topology. SEC Bench models the *network topology*, which defines a connectivity graph between nodes together with the properties of these connections. It instantiates this characteristic through the `network` section in its configuration file. Listing 1.1 shows an example configuration of entries to define the network topology parameters. In this example, the topology is specified in `custom` mode, and the link between `node1` and `node2` is provided along with its `latency` and `bandwidth` properties. Users can also select a predefined topology template, such as `line`, `tree`, or `ring`, which we present the complete set in the open source repository [53]. SEC Bench constructs the environment shown in Fig. 1 based on the hardware and network configurations specified in Listing 1.1.

3.2 Benchmarking Workflow and Components

SEC Bench has a modular architecture in which each framework component has a well-defined role in the execution of a benchmark run, and users can configure these components independently. Fig. 2 illustrates the SEC Bench’s benchmarking flow, with data generation (i.e., Benchmark Driver) on the left, the System Under Test (SUT) in the middle, and Performance Tracking on the right. SEC Bench configures the hardware ① and network ② characteristics to construct the benchmarking setup and emulate the hardware and network characteristics of the SEC continuum. As part of the benchmarking setup, it deploys the SUT as containerized hosts in the emulated environment. On the left-hand side of Fig. 2

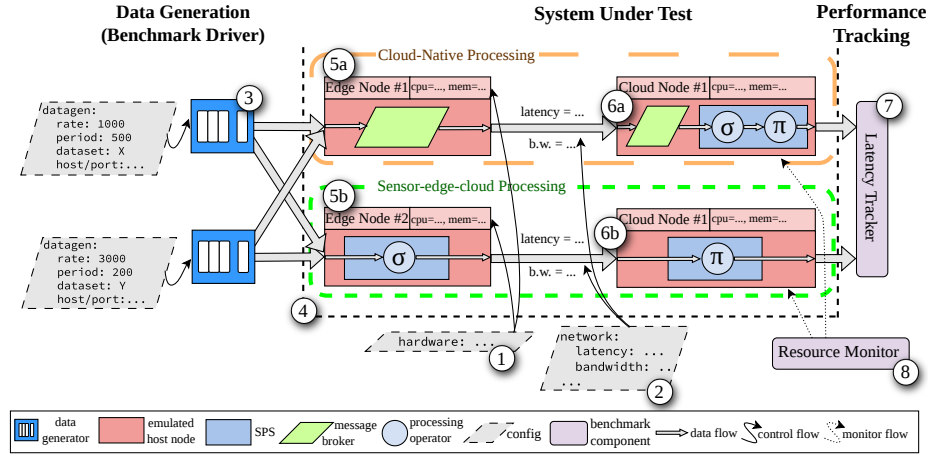


Fig. 2: Benchmarking flow of SEC Bench

③, instances of the data generator publish data into the SUT sources in parallel by following a data generation configuration. As shown at ④, we decouple the benchmark driver from the SUT by deploying them in separate instances. This ensures that they use different resources, and therefore, the SUT’s performance is not affected by the benchmark driver’s operations.

After the data arrives at the SUT’s sources, the SUT begins processing it internally. In this context, modern SPSs typically follow two distinct computing paradigms for data processing. First, the *cloud-native* processing model relies on a message broker at the edge node (5a) to forward upstream data to the cloud without applying processing. At the cloud node (6a), the broker receives the incoming data and delivers it to the SPS for processing. Fig. 2 highlights this model with an orange dashed outline (— —). In contrast, the *SEC* processing model deploys its processing engine on the edge node (5b) to push down computation closer to data sources. As a result, no additional message broker is required, and processing can be performed on the edge (5b) in addition to cloud node (6b). Fig. 2 highlights this model with a green dotted outline (· · ·). In both cases, the SUT writes its output to a sink (7), where SECBench’s Latency Tracker derives the end-to-end latency of the events. Throughout the experiments, SECBench’s Resource Monitor continuously tracks resource utilization on all deployed nodes (8). Together, these steps define a benchmark run of the SECBench: the framework constructs the configured environment, deploys the SUT and benchmark components, generates the workload, and collects the metrics. In the following, we describe the components of SECBench, and show how they interact with the SUT.

Data Generator. SECBench incorporates a configurable data generator that transforms input datasets into streaming workloads. Users can configure the temporal properties of the generated data streams, such as the input rate, period, and duration, as well as the workload characteristics, such as the selected dataset and the data generation pattern. The data generator can also publish data to multiple SUT sources in parallel by using the configured host and port endpoints. This allows SECBench to model both centralized ingestion and distributed data generation from multiple sources.

The data generator supports both statistically driven and trace-driven data generation patterns by overriding the existing timestamps of the input dataset. The statistically driven patterns, such as uniform, Poisson, and Zipfian, cover fundamental workload behaviors in a predictable and reproducible way. The trace-driven patterns use real sensor traces, including GSM, humidity, and barometric pressure measurements, which cover a range of varying temporal characteristics observed in real-world sensor streams [15]. We selected these patterns to cover a representative range of data characteristics relevant to stream processing benchmarks, while keeping the pattern space manageable. We provide the full configuration schema and parameters for patterns in the open-source repository [53].

SECBench loads input data into in-memory buffers before the experiments to avoid the disk access as a bottleneck [33]. It assigns instantaneous event-time timestamps while publishing the events. However, when the same input data is replayed multiple times, the original timestamps can no longer be reused. Therefore, SECBench uses a dedicated writer thread to overwrite the timestamp field of each event with wall-clock time. Furthermore, to enable parallel data generation to multiple SUT sources without performance degradation, SECBench uses

non-blocking sockets, in which each connection handles data publication in a separate OS thread. These threads share a pool of read-only buffers, that avoid redundant data copies and eliminate contention. SECBench continuously monitors the achieved generation rate during each run, and flags experiments when the generator fails to produce the configured data rate.

Latency Tracker. SECBench employs a *Latency Tracker* to obtain end-to-end latency measurements without instrumenting the SUT. The tracker observes the SUT output at the sink, and assigns a sink-arrival timestamp to each incoming event. Then it derives each event’s *event-time latency* using the difference between the sink-arrival timestamp and the event timestamp (issued by the Data Generator). For workloads with high output rates, the tracker can sample output events at a configurable rate to reduce the measurement overhead, while preserving the same timestamping procedure for sampled events. For the latency measurement, the SUT must preserve the event timestamp issued by the Data Generator, or provide an equivalent event-time reference for outputs produced by stateful operators. Sec. 4.1 defines the latency metric in the SEC setting, and describes how SECBench handles filtering and stateful operators.

Resource Monitor. Analyzing the performance of an SPS across the SEC continuum requires monitoring resource utilization on heterogeneous devices with varying computing capabilities. Since SECBench deploys nodes as containerized hosts, it is possible to track the resource usage of individual hosts via Docker API stats. SECBench provides a *Resource Monitor* that continuously observes the CPU, main memory, and storage usage of individual nodes by periodically sampling the corresponding metrics. This enables the identification of resource bottlenecks across the continuum, and supports a more fine-grained analysis.

Monitoring network utilization between nodes is important for identifying network bottlenecks and interpreting end-to-end performance results. Resource Monitor observes the bandwidth utilization of each inter-node link throughout the experiments, and makes the measurements available either as log files or via an interface for real-time analysis. We present the resource utilization metrics in detail in Sec. 4.

3.3 Environment Construction

Supporting distributed deployments in benchmarking remains an active area of research [42, 59]. SECBench contributes to this area by providing reproducible, container-based emulation of SEC deployments. It constructs SEC deployments using Containernet, which runs Docker containers as hosts within emulated network topologies [40]. During deployment, SECBench translates the configured hardware and network characteristics into connected containerized hosts with resource limits. Since this emulation runs applications on the Linux kernel and network stack, systems in such deployments exhibit performance comparable to production environments, as long as the host computer has sufficient resources.

3.4 SUT Configuration

SECBench treats an SPS as SUT if it can be deployed as a Dockerized application in the benchmarking environment. It assumes well-defined input endpoints (i.e., sources) where the SUT ingests generated events, and an output endpoint (i.e., sink) where the SUT outputs its results. The sources of the SUT are connected to SECBench’s data generator instances, while the SUT’s sink is connected to the Latency Tracker. For latency computation, the SUT’s output must preserve the event timestamp issued by the Data Generator, which enables event-time latency computation at the sink. In order to integrate a new SUT, users provide its Docker image, connect sources to the data generator and sink to the latency tracker. Overall, users can deploy and benchmark different SPSs under identical and reproducible conditions, while SECBench facilitates the deployment of the system, runs experiments, and collects performance metrics.

4 Evaluation

In this section, we evaluate SECBench as a benchmarking framework. We first define the evaluation metrics supported by the framework (Sec. 4.1), then introduce two real-world use cases to demonstrate the applicability of SECBench (Sec. 4.2 and Sec. 4.3), and finally discuss the overhead and limitations of SECBench (Sec. 4.4).

4.1 Evaluation Metrics

Benchmarking an SPS in the SEC continuum requires evaluation metrics that both assess stream processing performance and capture the characteristics of the distributed deployments. We use *latency*, *throughput*, and *resource utilization*, and adapt them to account for distributed settings. These metrics, together, provide a basis for evaluating SPS behavior operating across the SEC continuum. In the following, we define these metrics in detail.

Latency. We use *event-time latency* in SECBench because it provides an end-to-end, user-centric view that includes queuing and network delays, that reflects realistic deployments [24, 36]. In SEC environments, the source of data is often not a single instance, but instead gathered from multiple devices in different locations [56]. This can make it difficult to determine which data source becomes the bottleneck, and increases the overall latency of events during analysis. To overcome this, SECBench’s data generator tags each event with the identity of its source. In this way, we can distinguish events based on their data source and analyze the corresponding data paths through the SUT.

We formulate event-time latency as shown in Eq. (1). Let $S = \{s_1, \dots, s_m\}$ denote a set of data sources, and let e_{i,s_j} be an event i generated by the source $s_j \in S$. Then, $t_{event}(e_{i,s_j})$ represents the event time of e_{i,s_j} . Similarly, $t_{sink}(e_{i,s_j})$ is the time when the corresponding result of e_{i,s_j} arrives at the sink. Therefore, event-time latency is the time between the sink time (i.e., $t_{sink}(e_{i,s_j})$) and the event time (i.e., $t_{event}(e_{i,s_j})$). For deployments with multiple data sources, we compute event-time latency for each output event and aggregate these results

across all the sources. We report average and percentile latencies for these aggregated results and also provide a per-source breakdown for fine-grained analysis.

$$L_{event}(e_{i,s_j}) = t_{sink}(e_{i,s_j}) - t_{event}(e_{i,s_j}) \quad (1)$$

Eq. (1) applies to operators that preserve one-to-one correspondence between input and output events. However, many streaming workloads contain operators that break this correspondence. For filtering operators, some input events do not pass the filter and do not produce output events, so they have no sink arrival timestamp. We exclude these filtered events from the calculation and only calculate latency for the unfiltered results. In the case of stateful operators such as windowed aggregations or joins, a single output event may originate from multiple input events. In this case, we follow the approach proposed by Karimov et al., and use the latest event time among these inputs as the reference point [30]. Eq. (2) formulates the latency calculation for stateful operators. From an output event e_o that resulted from a set of input events $I(e_o)$, the event time latency is the elapsed time between its sink arrival time ($t_{sink}(e_o)$) and the latest event time among the contributing input events ($\max_{e_i \in I(e_o)} t_{event}(e_i)$).

$$L_{event}(e_o) = t_{sink}(e_o) - \max_{e_i \in I(e_o)} t_{event}(e_i) \quad (2)$$

Throughput. We use *throughput* to measure the processing capability of an SPS under load, which is defined as the number of events processed by an SPS per unit time, typically per second [59]. We measure throughput only in the range where the SUT can keep up with the incoming data stream. As we increase the input rate, the SUT eventually reaches a limit beyond which it cannot process the incoming data stream without backpressure. Once the input rate exceeds this limit, events accumulate in internal queues and the latency increases [37]. This limit is referred to as the *sustainable throughput* [11, 30]. We calculate the sustainable throughput of SUTs during our experiments and report it in the results.

Resource Utilization. In the SEC continuum, heterogeneous nodes process data under different hardware capacities and varying network conditions. As a result, performance bottlenecks may arise at individual nodes and between links connecting them. Since latency and throughput alone cannot reveal the causes of such bottlenecks, we collect node-level resource utilization metrics on each node for compute bottlenecks, as well as link-level network utilization on the links between them. At the node-level, SECBench monitors CPU, main memory, and storage utilization for each deployed node. CPU utilization helps to identify compute saturation, main memory utilization indicates memory pressure that may lead to paging or garbage collection overhead, and storage utilization helps to detect disk-related bottlenecks caused by buffering, spilling, or state persistence. SECBench collects these metrics individually for each node, and reports both per-node and aggregated statistics, such as the average and maximum among all deployed nodes.

For communication bottlenecks, SECBench monitors network utilization on each link between the deployed nodes. It measures the transmitted and received bytes over a sampling interval and relates this traffic volume to the configured link capacity. This metric shows how much of the available bandwidth is used during the experiment. Values closer to 100% indicate a link saturation, which can increase queuing delays and degrade end-to-end performance of the SUT. SECBench computes network utilization for each link between deployed nodes during the experiments and reports per-link, aggregate, and maximum-observed utilization to identify communication bottlenecks in the deployed topology.

4.2 Use Case: Smart Water Metering

The first real-world use case that we include in SECBench is based on a real LoRaWAN (Long Range Wide Area Network) deployment operated by UNIDATA S.p.A. in Italy to exemplify stream processing in the SEC continuum [18, 21]. Their infrastructure comprises more than 4000 end devices and 140 gateways across the operator’s service area with a central network server located in Rome. Water meters continuously generate live consumption readings and transmit them via LoRaWAN to gateways, which forward the collected data to the network server. Such data support applications such as dynamic resource management and leakage detection. However, the large number of devices and the continuous generation of sensor readings stress the limited network bandwidth and the response-time requirements of the services. This use case addresses these challenges by following a layered processing approach that spans edge and cloud resources. Rather than forwarding all raw sensor readings directly to the cloud, parts of the processing pipeline can be pushed closer to data sources. This allows SPSs to exploit available resources across the deployment in accordance with their optimization and placement strategies. As a result, SPSs can reduce network load and response times, and enable privacy-preserving data processing through local computations.

Dataset. The smart water metering dataset consists of time-series data collected from sensors attached to water meters. Each record contains water-demand and hydraulic-head measurements, together with spatial, temporal, and device-level metadata. Our workload uses the event time, the gateway identifier, the measured demand value, and the node type. The attribute `dateTime` denotes the time of a reading, while `gateway_id` identifies the gateway that forwards the reading. The attribute `demand_value` represents the measured water demand at a water-network node, and `node_type` indicates whether a water-network node injects (value 1) or absorbs (value 0) water in the infrastructure. We make the full schema available in our open-source repository [53].

Workload. The workload from the use case consists of two tasks: reducing redundant readings and detecting potential leaks. The Listing 1.2 combines these tasks and represents them as a query in CQL syntax [3]. It filters the input

stream to retain only positive readings using `demand_value`, groups events into 30-second sliding windows with a 1-second slide, and computes aggregations on `gateway_id`. The query uses `node_type` to separate water-injecting nodes from water-absorbing nodes. Within each window, the query computes the total injected and total absorbed demand and compares them. If these values differ, then the query emits the result which indicates a potential leakage.

Listing 1.2: Smart Water Metering Query

```
SELECT gateway_id, total_injected, total_absorbed,
       total_injected - total_absorbed AS leakage_gap
FROM (
  SELECT gateway_id,
         SUM(CASE WHEN node_type = 1 THEN demand_value ELSE 0 END) AS
           total_injected,
         SUM(CASE WHEN node_type = 0 THEN demand_value ELSE 0 END) AS
           total_absorbed
  FROM waterMeterReadings
  WINDOW SLIDING (SIZE 30 SECONDS, SLIDE 1 SECOND)
  WHERE demand_value > 0.0
  GROUP BY gateway_id
)
WHERE total_injected <> total_absorbed
```

Hardware and Network Settings. The smart water metering use case forms a LoRaWAN star-of-stars topology, with a single layer of end devices transmitting readings to gateways, and one layer of gateways connected to the network server. The links between end devices and gateways are wireless LoRaWAN connections, which we model with 125 kHz channel bandwidth and 250 ms latency [21]. In contrast, the links between gateways and the network server are 100 Mbps connections with latency of 10 ms. End devices are low-power meter devices without enough computing capability, therefore we model them as data sources in our benchmarking setup. Gateways form the first compute-capable layer and have an edge resource profile with 8 CPU cores, 16 GB of RAM, and 1 TB of storage. The central network server represents the cloud layer, which we model with 16 CPU cores, 128 GB of RAM, and 4 TB of storage.

Experiment. We run an experiment using the smart water metering use case by configuring SECBench’s environment characteristics based on the real deployment structure and the benchmark settings described above. In particular, we use the smart water metering dataset, execute the query from the use case, and parametrize the benchmarking environment with corresponding hardware and network settings.

We evaluate NebulaStream as a SEC SUT, and Flink with Kafka as a cloud-native SUT. For NebulaStream, we deploy its worker engines on all the deployed gateway and cloud nodes. Based on its optimization and placement strategy, NebulaStream can distribute the computation across these layers of the SEC continuum. For Flink, we deploy Kafka on gateway nodes to ingest and deliver input data to the cloud node, where we deploy Flink to process data. The goal of the experiment is not to compare or rank the systems, but to show how SECBench helps to investigate the performance effects caused by differ-

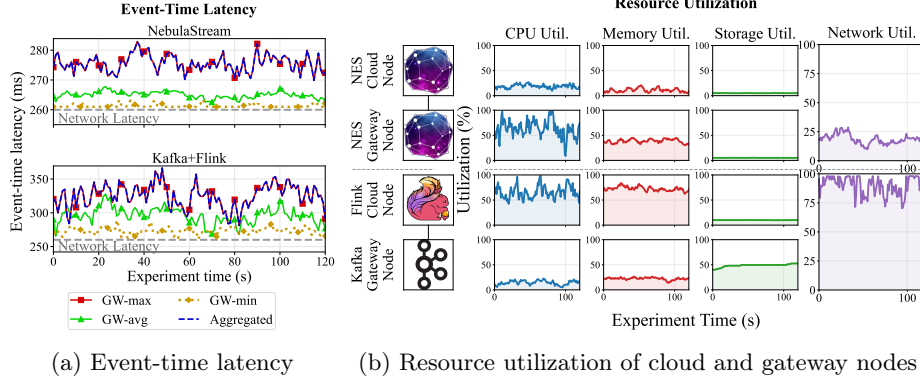


Fig. 3: Latency and resource utilization results of the experiment

ent processing models under the same workload and environment conditions. We run each experiment for 2 minutes and repeat it three times for each SUT. SECBench collects the evaluation metrics introduced in Sec. 4.1, i.e., latency, throughput, and resource utilization. We run the SUTs on a server with two 16-core Intel Xeon Gold 6326 CPUs, 1 TB memory, and 24 TB of storage, running Ubuntu 24.04.2 LTS. We additionally monitor the host-level utilization during the experiments to verify that the underlying host does not constrain the emulated environment.

Latency Analysis. Left-hand side of the Fig. 3 presents latency plots for NebulaStream and Flink+Kafka. For each SUT, we show the aggregated 95th percentile latency (P95), and event-time latencies for results associated with individual gateways. We present three representative gateways: Gateway-1 (GW-avg) has a latency close to the average, Gateway-104 (GW-min) has the overall minimum latency, and Gateway-6 (GW-max) exhibits the maximum observed latency during the experiments. Distinguishing latency results by gateway enables a more fine-grained analysis of the sources contributing to the overall latency. As shown in the figure, for both SUTs, GW-max has slightly higher latency than the aggregated P95 latency, indicating that GW-max constitutes the bottleneck. We also show the minimum achievable latency as a lower bound in the figure, represented by a dotted horizontal gray line (260 ms), which corresponds to the network delay from end-devices to the cloud. SECBench reveals that, in Flink+Kafka, the latency curves from individual gateways exhibit similar fluctuations throughout the experiment. When the latency of GW-max increases, this increase propagates to results from other gateways, because Kafka sends all raw data to Flink in the cloud for processing. SECBench also reveals that this effect is much less pronounced in NebulaStream. Since NebulaStream performs computation at the gateway level and forwards only the resulting outputs to the cloud node, a latency increase at one gateway has only a limited impact on the performance of other gateways.

Resource Utilization Analysis. To further investigate the cause of the bottleneck, we present the resource utilization measurements. The right-hand side of

the Fig. 3 shows the utilization of the nodes involved in the bottleneck path, i.e., GW-max and cloud nodes. In the upper part of the figure, we show a cloud node and a gateway node (GW-max), both of which run NebulaStream workers. In the lower part, we present the corresponding setup for Flink+Kafka, where Flink runs on the cloud node, and Kafka runs on the GW-max. For each setup, we report CPU, main memory, and storage utilization on the two nodes, as well as network utilization on the link between them.

SECBench reveals that NebulaStream actively uses the gateway node’s resources by pushing computation to GW-max and forwarding only the results to the cloud. While this approach prevents network saturation, it increases the CPU and memory utilization at the gateway because the query must retain state for a large number of events. In contrast, Kafka forwards all raw input data from GW-max to the cloud node for processing with Flink. As a result, Flink+Kafka saturates the network link between the gateway and the cloud, which increases the overall latency. We also observe increases in storage utilization at the Kafka gateway node during the periods of network saturation, which is because the Kafka buffers events when the link cannot transmit data at the incoming rate. Furthermore, during the experiments, the host CPU utilization remained below 75%, memory consumption remained below 600 GB of the available 1 TB, and no swapping occurred.

Throughput Analysis. We evaluate throughput in a separate experiment, since replaying the smart water metering dataset at its original pace does not create backpressure on the SUTs to measure the sustainable throughput. For each SUT, we increase the input rate until the SUT reaches the highest load that it can sustain without a persistent increase in the latency [30]. In this experimental setup, SECBench identifies sustainable throughput values of 2.3 M events/s for the SEC deployment with NebulaStream, and 1.2 M events/s for the cloud-native deployment with Flink+Kafka. These results illustrate how SECBench can quantify the sustainable throughput of different processing models under the same workload and environment configurations.

4.3 Use Case II: Traffic Congestion Detection

The second use case we include in SECBench is **traffic congestion detection** [35], a multimedia use case that processes video streams. In this use case, roadside cameras continuously produce video streams at geographically distributed locations and are connected to a central traffic management center of the New Jersey Department of Transportation (NJDOT) [39]. The objective of the use case is to detect congested road segments in near real time by analyzing video data from cameras. Since this requires processing large volumes of continuous data under latency and bandwidth constraints, the use case adopts distributed stream processing across the SEC continuum. Based on the capabilities and operator placement strategies of the SUT, processing can take place at the edge, in the cloud, or across both. Processing data closer to the sources and detecting congestion earlier supports timely traffic management decisions and helps to reduce delays in affected areas.

Dataset. We use the UA-Detrac dataset and combine it with the real-world scenario described above under the traffic congestion detection use case [58]. The dataset consists of traffic videos recorded at 25 frames per second with a frame resolution of 960x540 pixels, stored as JPEG images. Although the frame rate and resolution from cameras are uniform across the dataset, the byte size of individual frames may vary from 45 to 140 KB per frame between cameras due to differences in image content and JPEG compression. The videos capture real traffic scenarios from 24 different locations, including highways, traffic crossings, and T-junctions. In our setting, each video sequence from a different camera is replayed by SECBench’s data generator as a camera stream in the benchmarking environment.

Workload. The use case workload detects traffic congestion by running an ML inference model on continuous and distributed video stream data. The pretrained ML model `CAR_DETECT` processes each incoming frame and outputs a list of detected cars by assigning each detected event a unique identifier. Based on this, the query aggregates the number of detected cars over 30-second windows and groups them by `camera_id`. If the result of this aggregation exceeds 20, it indicates traffic congestion, and the query returns the result. The listing 1.3 shows the corresponding query in CQL syntax [3].

Listing 1.3: Traffic Congestion Detection Query

```
SELECT
  camera_id,
  COUNT(DISTINCT track_id) AS vehicle_count
FROM CAR_DETECT(raw_frames)
WINDOW SLIDING (SIZE 30 SECONDS, SLIDE 1 SECOND)
GROUP BY camera_id
HAVING
  COUNT(DISTINCT track_id) > 20
```

Hardware and Network Settings. The use case has a three-tier camera-edge-cloud topology, in which each camera is associated with a co-located edge server, and the edge servers are connected to a central traffic management system. Edge servers are cloudlet-class devices equivalent to an Intel NUC 7i7BNH with a 2-core CPU, 16 GB of RAM, and 1 TB of storage [35]. They are connected to the central traffic management system, which hosts substantially more powerful servers. These servers comprise two servers with 10-core E5-2630 v4 CPUs and two servers with 8-core E5-2620 v4 CPUs, and we model each with 64 GB of RAM [35]. We model the backhaul between the edge tier and the traffic management system with a bandwidth of 100 Mbps and an average network latency of 43 ms.

Experiment. We run the experiment using the traffic congestion detection use case by configuring SECBench according to the specification that we outlined above. We follow the same experimental methodology as in Use Case I (Section 4.2). Thus, we again evaluate NebulaStream as an SEC SUT, and Flink with Kafka as a cloud-native SUT. In contrast to the first use case, the input

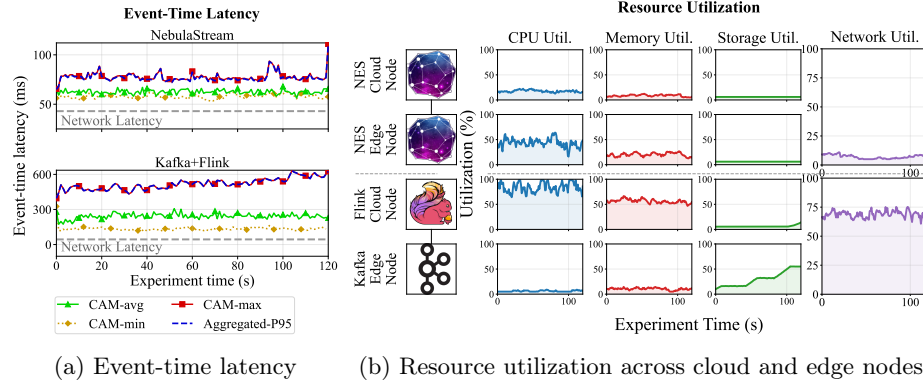


Fig. 4: Latency and resource utilization in the experiment for the traffic congestion detection use case.

events in this workload are video frames, which constitute a substantially larger event size. We additionally vary the frame rates of the individual camera streams to evaluate how the SUTs behave under different source rates.

Latency Analysis. We show the latency results in Fig. 4. CAM-avg represents the original frame rate from the dataset (25 FPS), CAM-min represents the lowest (5 FPS), and CAM-max represents the highest (60 FPS) frame rate. This per-camera breakdown enables us to show how different cameras with varying frame rates contribute to the end-to-end performance of the SUT.

For NebulaStream, the three camera streams exhibit relatively small differences in latency. This limited impact can be attributed to the distributed processing at the edge nodes. Each camera is associated with a dedicated edge server, which processes its input stream independently and has sufficient compute capacity. Therefore, the additional load introduced by a camera with a higher rate remains localized to its corresponding edge server and has only a limited effect on the end-to-end latency of the SUT.

The effect of high-rate cameras is more visible for Flink+Kafka. Since all camera streams are forwarded to the cloud and processed by the same centralized Flink deployment, changes in the load can affect the latency of all the camera streams. SECBench also shows that the aggregated latency follows the latency associated with CAM-max. This indicates that the highest-rate camera stream dominates the overall latency in both SUT deployments.

Resource Utilization Analysis. We further investigate the cause of the performance degradation by analyzing the resource utilization results collected by SECBench. For each SUT, we show the utilization of one representative edge node associated with CAM-max and the cloud node, as well as the network utilization of the link connecting them. Fig. 4 shows the latency and resource utilization results for the experiment. Overall, the results highlight the importance of operator placement for multimedia workloads in the SEC continuum. In particular, since NebulaStream places its ML inference operator on edge nodes, we observe higher

utilization of the edge node associated with CAM-max, and only the computed results are sent to the cloud node. In contrast, the Flink+Kafka deployment shows that more computing and memory resources are utilized on the cloud node because both the ML inference and window aggregation from every camera are executed centrally. However, the network is not the main bottleneck in this use case, as the transmitted camera streams do not saturate the available network bandwidth. SECBench reveals that the performance degradation in this use case stems from the shared computing resources of the centralized processing rather than the network saturation. We also observe increased storage utilization on the Kafka edge nodes, indicating that events accumulate in the broker when Flink cannot consume them at the incoming rate. These periods coincide with higher event-time latency, as an accumulating backlog in Kafka contributes to additional queuing delay.

Throughput Analysis. In a separate experiment, we increase the camera frame rate to measure the sustainable throughput of the SUTs. SECBench identifies that NebulaStream reaches up to 3.7 K events/s, whereas Flink+Kafka achieves 620 events/s. These numbers are lower than the throughput values from the first use case. The lower throughput is due to the larger event size and the compute-intensive ML inference performance for each event, both of which increase the processing cost per event.

4.4 Validation Experiment and Limitations

In this subsection, we validate SECBench by comparing a bare-metal setup with a corresponding environment deployed by SECBench. We then discuss the limitations of SECBench for benchmarking SPSs in distributed deployments.

Validation Experiment. We conduct a validation experiment by comparing an emulated setup deployed by SECBench with a bare-metal setup that replicates the same environment. The objective is to assess how closely SECBench reproduces the performance behavior of a real deployment and to quantify the overhead introduced by the emulated environment. To this end, we build a physical testbed comprising two Raspberry Pi devices, one desktop computer, and one rack-mounted server, which together form a minimal SEC continuum setup. The Raspberry Pis are connected to the desktop computer via Ethernet cables, and the desktop computer is connected to the server via a Gigabit Ethernet link. We use the dataset and query from smart water metering use case (see Sec. 4.2) as a representative workload for this setup. SECBench reproduces this environment by configuring the same hardware capacities and network characteristics in the emulated deployment. In both setups, the data generator of SECBench runs on a separate machine and sends input events to the Raspberry Pi devices, which act as SUT’s data sources. Similarly, we deploy the latency tracker of SECBench on the same external machine, which receives the output events from the server node, i.e., the SUT sink. This design ensures that data generation and latency measurement remain outside of the SUT, and do not interfere with its execution.

We evaluate NebulaStream as a SEC SUT and Flink+Kafka as a cloud-native SUT. For each SUT, we ensure the same operator placement, and use the same

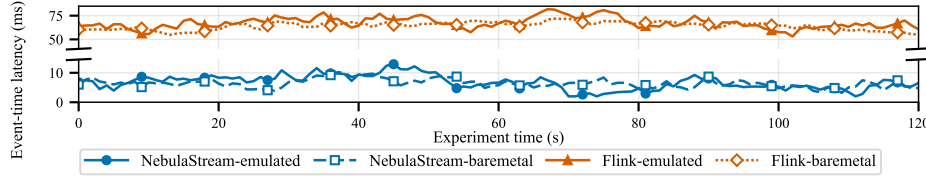


Fig. 5: Latency results of the validation experiment

dataset and query in both the bare-metal testbed and in the corresponding environment deployed by SECBench. Fig. 5 shows the latency results of the experiment. Performance results in emulated environments closely follow the bare-metal setup, with a relative latency deviations of up to 4% for Flink+Kafka and 7% for NebulaStream. This overhead comes from virtual networking and containerization, which slightly increases the end-to-end latency compared to bare-metal setup. We note that this validation covers the scale of physical testbed and does not assess the accuracy of SECBench at larger deployment scales.

Limitations. Even though SECBench enables reproducible benchmarking across the SEC continuum, it has some limitations. First, it emulates the target scale-out environment on a single scale-up machine. This requires reproducing the properties of a distributed target environment on a host with sufficient compute, memory, and storage resources to accommodate all the emulated nodes. Therefore, the size of the emulated environment is limited by the capacity of a single host system.

Furthermore, although the emulation allows configuring the deployed nodes for CPU cores, main memory, and storage, it does not reproduce the architectural differences of physical devices. For example, the Raspberry Pi nodes in our bare-metal setup have ARMv8 CPUs, but the corresponding emulated nodes run on x86-64 host architecture. Thus, the emulated nodes inherit the execution characteristics of the host machine. However, the small latency difference in Fig. 5 indicates that this difference does not dominate the performance results.

5 Related Work

Building on the benchmark taxonomy of Yue et al. [59], we categorize related work on SPS benchmarking into *(i)* benchmark specifications and suites, *(ii)* system-level evaluations, and *(iii)* benchmarking tools and frameworks.

Benchmark specifications and suites. These works define workloads, data generators, and metrics for evaluating SPSs, but do not model the deployment environment as a configurable property. Linear Road Benchmark [4] models an urban tolling scenario with continuous and historical queries. NEXMark [54] adapts the XMark benchmark to an auction-based application by providing a data generator and a set of benchmark queries. HiBench [28] and BigBench [19] include streaming queries as part of their broader focus on batch, analytical, and large-scale data processing. The Yahoo Streaming Benchmark (YSB) [10] models an advertisement analytics pipeline, and RIoT Bench [49] focuses on IoT stream

processing and provides a collection of IoT-oriented applications. TPCx-IoT [41] offers a standardized benchmark for IoT gateway systems, with an emphasis on ingestion throughput and price-performance. Theodolite [22] evaluates the scalability of cloud-native stream processing applications, and SProBench [32] targets Slurm-based HPC infrastructures.

System-level evaluations. Several studies use workloads to evaluate SPSs under different metrics and execution settings. Karimov et al. [30] emphasize sustainable throughput and the distinction between processing-time and event-time latency. Other studies evaluate SPSs in more specialized contexts, such as smart city IoT workloads [38], real-time IoT applications [37], advertisement analytics [47], abstraction layers for stream processing [25], hardware-aware stream processing [61], and large-scale data shuffling [23]. These studies mainly evaluate SPSs in predefined or centrally managed environments, and do not focus on constructing configurable SEC environments with heterogeneous devices and network links.

Benchmarking frameworks and tools. These works provide configurable execution environments and generate benchmarks. SPBench [17] supports the creation of SPS benchmarks with APIs and CLI tooling. PGVal [52] evaluates processing guarantees and fault-tolerance behavior. PDSPBench [1] targets parallel and distributed stream processing and evaluates how operator parallelism and resource configurations influence the performance on heterogeneous hardware. It is the closest to our work since it also treats heterogeneity as an important factor in SPS benchmarking. However, PDSPBench focuses on parallel dataflows over heterogeneous cluster resources. In contrast, SECBench treats sensor, edge, and cloud resources as configurable resources of the SUT deployment. It combines configurable SEC environments with workload generation, benchmark execution, and performance measurement. This enables reproducible comparisons between cloud-only and SEC processing models.

6 Conclusion

In this paper, we presented SECBench, an end-to-end benchmarking framework for evaluating SPSs in the SEC continuum. Existing benchmarks provide limited support for deployment environments that involve heterogeneous devices and network links, and geographically distributed data sources. SECBench addresses this gap by modeling hardware heterogeneity and network topology as configurable environment characteristics and using them to construct reproducible, container-based benchmark environments. It combines these environments with high-rate and parallel data generation, end-to-end latency tracking, and monitoring of resource utilization across deployed nodes and network links. By coordinating the environment setup, workload generation, SUT execution, and measurement, SECBench provides an end-to-end flow for benchmarking SPSs in the SEC continuum.

We demonstrated the applicability of SECBench with two real-world use cases with representative workload and deployment characteristics. The smart

water metering use case shows how SECBench can identify bottlenecks that emerge from constrained network bandwidths and different processing models. The traffic congestion detection use case demonstrates how SECBench reveals compute bottlenecks caused by central processing, even when the network is not the primary limitation. The results highlight that the performance of SPSs in the SEC depends on the system, workload, operator placement and the characteristics of the underlying environment. We also validated SECBench against a corresponding bare-metal setup. The results show that the emulated environment of the SECBench captures the performance behavior of the evaluated SPSs, while introducing only a small latency overhead due to containerization and virtual networking. Future work will focus on extending SECBench towards multi-host emulation and richer hardware models. Overall, SECBench provides a reproducible foundation for studying how characteristics of SEC deployments affect SPS performance, and for comparing cloud-native and SEC processing models under realistic conditions.

References

1. Agnihotri, P., Koldehofe, B., Heinrich, R., Binnig, C., Luthra, M.: Pdsp-bench: A benchmarking system for parallel and distributed stream processing. In: Technology Conference on Performance Evaluation and Benchmarking. pp. 1–23. Springer (2024)
2. Al-Dulaimy, A., Jansen, M., Johansson, B., Trivedi, A., Iosup, A., Ashjaei, M., Galletta, A., Kimovski, D., Prodan, R., Tserpes, K., et al.: The computing continuum: From iot to the cloud. *Internet of Things* **27**, 101272 (2024)
3. Arasu, A., Babu, S., Widom, J.: The cql continuous query language: semantic foundations and query execution. *The VLDB Journal* **15**(2), 121–142 (2006)
4. Arasu, A., Cherniack, M., Galvez, E., Maier, D., Maskey, A.S., Ryvkina, E., Stonebraker, M., Tibbetts, R.: Linear road: a stream data management benchmark. In: Proceedings of the Thirtieth international conference on Very large data bases-Volume 30. pp. 480–491 (2004)
5. Babcock, B., Babu, S., Datar, M., Motwani, R., Widom, J.: Models and issues in data stream systems. In: Proceedings of the twenty-first ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems. pp. 1–16 (2002)
6. Bhattacharya, R., Wood, T.: Load balancing and generalized split state reconciliation in event driven systems. In: 2024 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS). pp. 127–132. IEEE (2024)
7. Bonomi, F., Milito, R., Zhu, J., Addepalli, S.: Fog computing and its role in the internet of things. In: Proceedings of the first edition of the MCC workshop on Mobile cloud computing. pp. 13–16 (2012)
8. Bordin, M.V., Griebler, D., Mencagli, G., Geyer, C.F., Fernandes, L.G.L.: Dsp-bench: A suite of benchmark applications for distributed data stream processing systems. *IEEE Access* **8**, 222900–222917 (2020)
9. Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., Tzoumas, K.: Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering* **38**(4) (2015)
10. Chintapalli, S., Dagit, D., Evans, B., Farivar, R., Graves, T., Holderbaugh, M., Liu, Z., Nusbaum, K., Patil, K., Peng, B.J., et al.: Benchmarking streaming computation engines: Storm, flink and spark streaming. In: 2016 IEEE international

- parallel and distributed processing symposium workshops (IPDPSW). pp. 1789–1792. IEEE (2016)
11. Chu, Z., Yu, J., Hamdull, A.: Maximum sustainable throughput evaluation using an adaptive method for stream processing platforms. *IEEE Access* **8**, 40977–40988 (2020)
12. De Assuncao, M.D., da Silva Veith, A., Buyya, R.: Distributed data stream processing and edge computing: A survey on resource elasticity and future directions. *Journal of Network and Computer Applications* **103**, 1–17 (2018)
13. van Dongen, G., Steurtewagen, B., Van den Poel, D.: Latency measurement of fine-grained operations in benchmarking distributed stream processing frameworks. In: 2018 IEEE International Congress on Big Data (BigData Congress). pp. 247–250. IEEE (2018)
14. Ecker, R., Karagiannis, V., Sober, M., Ebrahimi, E., Schulte, S.: Latency-aware placement of stream processing operators. *European Conference on Parallel Processing* pp. 30–41 (2023)
15. Flores, H., Hui, P., Tarkoma, S., Li, Y., Anagnostopoulos, T., Kostakos, V., Luo, C., Su, X.: Sensorclone: a framework for harnessing smart devices with virtual sensors. In: *Proceedings of the 9th ACM Multimedia Systems Conference*. pp. 328–338 (2018)
16. Fragkoulis, M., Carbone, P., Kalavri, V., Katsifodimos, A.: A survey on the evolution of stream processing systems. *The VLDB Journal* **33**(2), 507–541 (2024)
17. Garcia, A.M., Griebler, D., Schepke, C., Fernandes, L.G.: Spbench: a framework for creating benchmarks of stream processing applications. *Computing* **105**(5), 1077–1099 (2023)
18. Garlisi, D., Martino, A., Zouwayhed, J., Pourrahim, R., Cuomo, F., et al.: Exploratory approach for network behavior clustering in lorawan. *Journal of Ambient Intelligence and Humanized Computing* pp. 1–15 (2021)
19. Ghazal, A., Rabl, T., Hu, M., Raab, F., Poess, M., Crolotte, A., Jacobsen, H.A.: Bigbench: Towards an industry standard benchmark for big data analytics. In: *Proceedings of the 2013 ACM SIGMOD international conference on Management of data*. pp. 1197–1208 (2013)
20. Gkonis, P., Giannopoulos, A., Trakadas, P., Masip-Bruin, X., D’Andria, F.: A survey on iot-edge-cloud continuum systems: Status, challenges, use cases, and open issues. *Future Internet* **15**(12), 383 (2023)
21. Haxhibeqiri, J., De Poorter, E., Moerman, I., Hoebeke, J.: A survey of lorawan for iot: From technology to application. *Sensors* **18**(11), 3995 (2018)
22. Henning, S., Hasselbring, W.: Theodolite: Scalability benchmarking of distributed stream processing engines in microservice architectures. *Big Data Research* **25**, 100209 (2021)
23. Henning, S., Vogel, A., Leichtfried, M., Ertl, O., Rabiser, R.: Shufflebench: A benchmark for large-scale data shuffling operations with distributed stream processing frameworks. In: *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*. pp. 2–13 (2024)
24. Hesse, G.: A benchmark for enterprise stream processing architectures. Ph.D. thesis, Universität Potsdam (2022)
25. Hesse, G., Matthies, C., Glass, K., Huegle, J., Uflacker, M.: Quantitative impact evaluation of an abstraction layer for data stream processing systems. In: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS). pp. 1381–1392. IEEE (2019)

26. Hesse, G., Matthies, C., Perscheid, M., Uflacker, M., Plattner, H.: Espbench: The enterprise stream processing benchmark. In: *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. pp. 201–212 (2021)
27. Hirsch, M., Mateos, C., Zunino, A.: Augmenting computing capabilities at the edge by jointly exploiting mobile devices: A survey. *Future Generation Computer Systems* **88**, 644–662 (2018)
28. Huang, S., Huang, J., Dai, J., Xie, T., Huang, B.: The hibench benchmark suite: Characterization of the mapreduce-based data analysis. In: *2010 IEEE 26th International conference on data engineering workshops (ICDEW 2010)*. pp. 41–51. IEEE (2010)
29. Isah, H., Abughofa, T., Mahfuz, S., Ajerla, D., Zulkernine, F., Khan, S.: A survey of distributed data stream processing frameworks. *IEEE Access* **7**, 154300–154316 (2019)
30. Karimov, J., Rabl, T., Katsifodimos, A., Samarev, R., Heiskanen, H., Markl, V.: Benchmarking distributed stream data processing systems. In: *2018 IEEE 34th international conference on data engineering (ICDE)*. pp. 1507–1518. IEEE (2018)
31. Khalyeyev, D., Bureš, T., Hnětynka, P.: Towards characterization of edge-cloud continuum. In: *European Conference on Software Architecture*. pp. 215–230. Springer (2022)
32. Kulkarni, A.D., Ghiasvand, S.: Sprobench: Stream processing benchmark for high performance computing infrastructure. In: *European Conference on Parallel Processing*. pp. 268–282. Springer (2025)
33. Lerner, A., Jasny, M., Jepsen, T., Binnig, C., Cudré-Mauroux, P.: Dbms annihilator: a high-performance database workload generator in action. *Proceedings of the VLDB Endowment* **15**(12), 3682–3685 (2022)
34. Liu, F., Huang, J., Wang, X.: Joint task offloading and resource allocation for device-edge-cloud collaboration with subtask dependencies. *IEEE Transactions on Cloud Computing* **11**(3), 3027–3039 (2023)
35. Liu, G., Shi, H., Kiani, A., Khreishah, A., Lee, J., Ansari, N., Liu, C., Yousef, M.M.: Smart traffic monitoring system using computer vision and edge computing. *IEEE Transactions on Intelligent Transportation Systems* **23**(8), 12027–12038 (2021)
36. Lohrmann, B.: Massively parallel stream processing with latency guarantees. Ph.D. thesis, Technische Universitaet Berlin (Germany) (2016)
37. Mishra, L., Varma, S., et al.: Performance evaluation of real-time stream processing systems for internet of things applications. *Future Generation Computer Systems* **113**, 207–217 (2020)
38. Nasiri, H., Nasehi, S., Goudarzi, M.: Evaluation of distributed stream processing frameworks for iot applications in smart cities. *Journal of Big Data* **6**(1), 52 (2019)
39. New Jersey Department of Transportation: Traffic cameras, traveler info. <https://dot.nj.gov/transportation/traffic/cameras/> (2020), official NJDOT traffic cameras page. Accessed: 2026-04-13
40. Peuster, M., Karl, H., van Rossem, S.: Medicine: Rapid prototyping of production-ready network services in multi-pop environments. In: *2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. pp. 148–153 (Nov 2016)
41. Poess, M., Nambiar, R., Kulkarni, K., Narasimhadevara, C., Rabl, T., Jacobsen, H.A.: Analysis of tpcx-iot: The first industry standard benchmark for iot gateway systems. In: *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. pp. 1519–1530. IEEE (2018)

42. Qu, L., Wang, Q., Chen, T., Li, K., Zhang, R., Zhou, X., Xu, Q., Yang, Z., Yang, C., Qian, W., et al.: Are current benchmarks adequate to evaluate distributed transactional databases? *BenchCouncil Transactions on Benchmarks, Standards and Evaluations* **2**(1), 100031 (2022)
43. Rao, K., Coviello, G., Chakradhar, S.: Clap: Cost and latency-aware placement of microservices on the computing continuum. In: *2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW)*. pp. 68–77. IEEE (2024)
44. Raspberry Pi Ltd: Raspberry pi computer hardware. Raspberry Pi Documentation, <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>, accessed: 2026-02-21
45. Samie, F., Bauer, L., Henkel, J.: Iot technologies for embedded computing: A survey. In: *Proceedings of the Eleventh IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis*. pp. 1–10 (2016)
46. Satyanarayanan, M.: The emergence of edge computing. *computer* **50**(1), 30–39 (2017)
47. Shahverdi, E., Awad, A., Sakr, S.: Big stream processing systems: an experimental evaluation. In: *2019 IEEE 35th International Conference on Data Engineering Workshops (ICDEW)*. pp. 53–60. IEEE (2019)
48. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. *IEEE internet of things journal* **3**(5), 637–646 (2016)
49. Shukla, A., Chaturvedi, S., Simmhan, Y.: Riotbench: An iot benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience* **29**(21), e4257 (2017)
50. Sinha, S.: State of iot 2025: Number of connected iot devices growing 14% to 21.1 billion globally. *IoT Analytics* (Oct 2025), <https://iot-analytics.com/number-connected-iot-devices/>
51. Stonebraker, M., Çetintemel, U., Zdonik, S.: The 8 requirements of real-time stream processing. *ACM Sigmod Record* **34**(4), 42–47 (2005)
52. Tahir, J., Mayer, R., Doblander, C., Jacobsen, H.A.: How reliable are streams? end-to-end processing-guarantee validation and performance benchmarking of stream processing systems. *Proceedings of the VLDB Endowment* **18**(3), 585–598 (2024)
53. Tekdogan, T.: SECBench. <https://github.com/nebulastream/secbench> (2026), gitHub repository, accessed 2026-08-11
54. Tucker, P., Tufte, K., Papadimos, V., Maier, D.: Nexmark—a benchmark for queries over data streams (draft). Technical report (2008)
55. Vantage: r7g.metal pricing and specs - vantage. Vantage Instances (AWS EC2), <https://instances.vantage.sh/aws/ec2/r7g.metal?currency=USD>, accessed: 2026-02-21
56. Wang, T., Liang, Y., Shen, X., Zheng, X., Mahmood, A., Sheng, Q.Z.: Edge computing and sensor-cloud: Overview, solutions, and directions. *ACM computing surveys* **55**(13s), 1–37 (2023)
57. Wang, Y., Yang, C., Lan, S., Zhu, L., Zhang, Y.: End-edge-cloud collaborative computing for deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials* **26**(4), 2647–2683 (2024)
58. Wen, L., Du, D., Cai, Z., Lei, Z., Chang, M.C., Qi, H., Lim, J., Yang, M.H., Lyu, S.: Ua-detrac: A new benchmark and protocol for multi-object detection and tracking. *Computer Vision and Image Understanding* **193**, 102907 (2020)
59. Yue, W., Boissier, M., Rabl, T.: A survey of stream processing system benchmarks. In: *Technology Conference on Performance Evaluation and Benchmarking*. pp. 24–43. Springer (2024)

60. Zaharia, M., Das, T., Li, H., Hunter, T., Shenker, S., Stoica, I.: Discretized streams: Fault-tolerant streaming computation at scale. In: Proceedings of the twenty-fourth ACM symposium on operating systems principles. pp. 423–438 (2013)
61. Zeuch, S., Breß, S., Rabl, T., Del Monte, B., Karimov, J., Lutz, C., Renz, M., Traub, J., Markl, V.: Analyzing efficient stream processing on modern hardware. *Proc. VLDB Endow.* **12**(5), 516–530 (2019)
62. Zhou, F., Chai, Y.: Near-sensor and in-sensor computing. *Nature Electronics* **3**(11), 664–671 (2020)